STUDENT WORKBOOK

The DSA Bug Library

39 worked problems with full solutions for Parts 0 to 12: how to spot common bugs in Data Structures & Algorithms, and how to fix them.

Prepared by Sumedh Manwatkar · Technical Trainer, GLA University

How to use this workbook

1. **Read the catalogue** at the start of each part. It lists the five common bugs, how each one shows itself, and the fix pattern.
2. **Try each problem first.** Read the task and the buggy code, then cover the solution and find the bug yourself.
3. **Check the solution.** It explains why the code fails (with a trace), shows the fixed code with the changed lines marked ✔, and gives a one-line rule to remember.
4. **Match the shape, not the wording.** The same mistake appears in many disguises. Name the pattern, and the next bug will be easy to spot.

How each problem is laid out

✗ Buggy code | Test: input, expected, got | ✔ Solution: why it fails, fixed code | Remember

All code is Java and every example was compiled and run: the buggy version really produces the wrong output shown, and the fixed version produces the expected one. Difficulty tags: **Easy**, **Medium**, **Hard**.

Contents

0 Six Meta-Patterns Behind Almost Every Bug p. 2
1 Arrays & Strings p. 4
2 Two Pointers & Sliding Window p. 6
3 Binary Search p. 8
4 Sorting p. 11
5 Linked Lists p. 13
6 Stacks & Queues p. 16
7 Trees & Binary Search Trees p. 18
8 Graphs (BFS/DFS) p. 21
9 Dynamic Programming p. 23
10 Recursion & Backtracking p. 25
11 Greedy Algorithms p. 27
12 Hashing & Hash Maps p. 30

THE GOLDEN RULE

Before fixing anything, say out loud what the correct value **should be** at the smallest possible input. Then find the exact line where the code's actual behavior **diverges** from that sentence. That line is almost always the bug.

0 PART 0
Six Meta-Patterns Behind Almost Every Bug

Almost every bug is one of six shapes. Learn to name the shape and you can fix bugs in any data structure.

Bug	How to spot it	Fix pattern
Off-by-one	Index/bound is one too many or few	Trace n=0,1,2 by hand; adjust the bound
Wrong comparison direction	Result is the opposite of what's wanted	State what should win, then check the operator
Missing/wrong base case	Crashes on smallest input, or answer is offset	Match the base case to the literal empty case
State not reset/updated	Works once, then gives stale results	Find every piece of state and confirm it updates
Reference instead of snapshot	Stored results all end up identical	Copy the value when you store it, not a live reference
Order-of-operations	A value is used after being overwritten/freed	Save what you're about to overwrite first

Problem 0.1 Find the Largest Number

Wrong comparison direction · Easy

Task. Return the largest value in an array. For {3, 9, 4} the answer must be 9.

✗ Buggy code. Find the bug before you read the solution.

```
static int largest(int[] a) {
    int best = a[0];
    for (int x : a)
        if (x < best) best = x;
    return best;
}
```

Test: largest({3, 9, 4}) → **Expected:** 9 **Got:** 3

Solution 0.1 Find the Largest Number

Why it fails. The program gives the opposite of what we want, which is the signature of a reversed comparison. Trace it: best = 3; 9 < 3 is false, so 9 is ignored; 4 < 3 is false. The loop can only ever pick a smaller value, so it finds the minimum.

✓ Fixed code (changed lines are marked ✓)

```
static int largest(int[] a) {
    int best = a[0];
    for (int x : a)
        if (x > best) best = x;   // ✓ bigger value must win
    return best;
}
```

Remember: Before checking the operator, say in words who should win ("the bigger number wins"). Then choose > or < to match that sentence.

Problem 0.2 Power of a Number

Missing / wrong base case · Easy

Task. Compute base^n recursively (n ≥ 0). power(2, 5) = 32 and power(2, 0) = 1.

✗ **Buggy code. Find the bug before you read the solution.**

```
static int power(int base, int n) {
    if (n == 1) return base;
    return base * power(base, n - 1);
}
```

Test: power(2, 5) and power(2, 0) → **Expected:** `32 1` **Got:** `Exception: StackOverflowError`

Solution 0.2 Power of a Number

Why it fails. power(2, 5) works because it reaches n == 1. But power(2, 0) never hits that base case: n goes 0, -1, -2, ... and the recursion never stops until the call stack overflows. The base case was written for n = 1, not for the smallest legal input n = 0.

✓ **Fixed code (changed lines are marked ✓)**

```
static int power(int base, int n) {
    if (n == 0) return 1;   // ✓ base case = the smallest input
    return base * power(base, n - 1);
}
```

Remember: Match the base case to the literal smallest input. Ask: what is the answer when n = 0? (Anything to the power 0 is 1.)

Problem 0.3 Fibonacci Without Recursion

Order-of-operations

Medium

Task. Return the n-th Fibonacci number using two variables (0, 1, 1, 2, 3, 5, 8, ...). fib(6) must be 8.

✗ **Buggy code. Find the bug before you read the solution.**

```
static int fib(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        a = b;
        b = a + b;
    }
    return a;
}
```

Test: fib(6) → **Expected:** `8` **Got:** `32`

Solution 0.3 Fibonacci Without Recursion

Why it fails. Line 1 of the loop overwrites a with b, so on the next line b = a + b actually computes b + b, doubling instead of adding. The old value of a was needed but already destroyed: 1, 2, 4, 8, 16, 32.

✔ Fixed code (changed lines are marked ✔)

```
static int fib(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        int next = a + b;   // ✔ save before overwriting
        a = b;
        b = next;
    }
    return a;
}
```

Remember: If a value is used after it has been overwritten, save it first in a temporary variable (next) and only then update the variables.

1 PART 1 Arrays & Strings

Arrays and strings fail at their edges: the first element, the last element, the empty input and very large values.

Bug	How to spot it	Fix pattern
Off-by-one loop bound	Misses the last element, or reads arr[n]	i < n for all n elements; i <= n only for a virtual slot
Empty input crash	Wrong answer on [] or "" before real logic runs	Check empty input first, return the defined answer
Mutating while iterating	Elements skipped/double-visited after a splice	Iterate backwards when removing, or build a new array
Wrong mutability assumption	Input array changes unexpectedly (or doesn't)	Copy first if input must stay unmodified
Integer overflow on large sums	Correct on small inputs, wrong on large ones	Use a wider type (long) for running sums

Problem 1.1 Remove Even Numbers

Mutating while iterating · Medium

Task. Remove every even number from a list, in place. [2, 4, 6, 7] must become [7].

✗ Buggy code. Find the bug before you read the solution.

```
static void removeEvens(List<Integer> list) {
    for (int i = 0; i < list.size(); i++)
        if (list.get(i) % 2 == 0) list.remove(i);
}
```

Test: removeEvens([2, 4, 6, 7]) → **Expected:** `[7]` **Got:** `[4, 7]`

Solution 1.1 Remove Even Numbers

Why it fails. When index 0 (value 2) is removed, every later element slides one place left: 4 moves into index 0. The loop then moves to index 1, so 4 is never examined. Removing while walking forward skips the neighbour of every removed element.

✔ Fixed code (changed lines are marked ✔)

```
static void removeEvens(List<Integer> list) {
    for (int i = list.size() - 1; i >= 0; i--)   // ✔ walk backwards
        if (list.get(i) % 2 == 0) list.remove(i);
}
```

Remember: When removing from a list inside a loop, iterate backwards (removals only shift elements you have already visited) or build a new list.

Problem 1.2 Second Largest Mark

Wrong mutability assumption · Easy

Task. Return the second largest mark. The original array of marks must stay in its original order, so {40, 90, 70} gives 70 and the array is unchanged.

✗ Buggy code. Find the bug before you read the solution.

```
static int secondLargest(int[] a) {
    Arrays.sort(a);
    return a[a.length - 2];
}
```

Test: secondLargest(marks), then print marks → **Expected:** `70 [40, 90, 70]` **Got:** `70 [40, 70, 90]`

Solution 1.2 Second Largest Mark

Why it fails. The answer 70 is right, but Arrays.sort sorts the array it is given, in place. Java passes the array reference, so the caller's marks array was silently reordered as a side effect.

✔ Fixed code (changed lines are marked ✔)

```
static int secondLargest(int[] a) {
    int[] copy = Arrays.copyOf(a, a.length);   // ✔ work on a copy
    Arrays.sort(copy);
    return copy[copy.length - 2];
}
```

Remember: Arrays and lists are passed by reference. If the input must stay unchanged, copy it first (Arrays.copyOf) and modify the copy.

Problem 1.3 Count Adjacent Equal Letters

Off-by-one loop bound · Easy

Task. Count how many neighbouring character pairs are equal. In "aabbbc" the pairs are aa, bb, bb, so the answer is 3.

✗ Buggy code. Find the bug before you read the solution.

```
static int countAdjacentEqual(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++)
        if (s.charAt(i) == s.charAt(i + 1)) count++;
    return count;
}
```

Test: countAdjacentEqual("aabbbc") → **Expected:** `3` **Got:** `Exception: StringIndexOutOfBoundsException`

Solution 1.3 Count Adjacent Equal Letters

Why it fails. The code reads charAt(i + 1). When i is the last index (5), i + 1 is 6, which is outside the string of length 6. Comparing a character with its right neighbour needs a right neighbour, so the loop must stop one step earlier.

✔ Fixed code (changed lines are marked ✔)

```
static int countAdjacentEqual(String s) {
    int count = 0;
    for (int i = 0; i < s.length() - 1; i++)   // ✔ last pair starts at n-2
        if (s.charAt(i) == s.charAt(i + 1)) count++;
    return count;
}
```

Remember: If the loop body touches i + 1, the bound is length - 1. Trace n = 0, 1, 2 by hand to confirm.

2 PART 2 Two Pointers & Sliding Window

A window or a pair of pointers is state. Whenever a pointer moves, every piece of state must move with it.

Bug	How to spot it	Fix pattern
if instead of while when shrinking	Window shrinks one step even when it needs several	Use while (invalid) { shrink(); }, never a single if
Off-by-one window size	Reported length/sum is one short or too many	Size is right - left + 1, not right - left
Forgetting to undo on shrink	Frequency map/sum stays wrong after left moves	Undo the exact contribution when an element leaves
Pointer never advances	Program hangs on certain inputs	Guarantee every branch advances at least one pointer
State not reset between windows	A later window's answer is contaminated	Reset counters/maps at the start of each scan

Problem 2.1 Longest Substring Without Repeats

Forgetting to undo on shrink · Medium

Task. Return the length of the longest substring with all different characters. For "abcabcbb" the answer is 3 ("abc").

✗ Buggy code. Find the bug before you read the solution.

```
static int longestUnique(String s) {
    Set<Character> seen = new HashSet<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        while (seen.contains(s.charAt(right)))
            left++;
        seen.add(s.charAt(right));
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Test: longestUnique("abcabcbb") → **Expected:** `3` **Got:** `never finishes (infinite loop)`

Solution 2.1 Longest Substring Without Repeats

Why it fails. At right = 3 the new 'a' is already in the set, so the while loop starts moving left. But left++ never removes anything from the set, so the condition stays true forever: an infinite loop. The window moved but the set (its state) did not.

✓ Fixed code (changed lines are marked ✓)

```
static int longestUnique(String s) {
    Set<Character> seen = new HashSet<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        while (seen.contains(s.charAt(right))) {
            seen.remove(s.charAt(left));   // ✓ undo what leaves the window
            left++;
        }
        seen.add(s.charAt(right));
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Remember: Whenever the window shrinks, undo the exact contribution of the element that leaves (remove it from the set / subtract it from the sum / decrement the count).

Problem 2.2 Count Pairs With a Given Sum

Pointer never advances · Medium

Task. In a sorted array of distinct numbers, count the pairs that add up to the target. {1, 2, 3, 4, 5} with target 6 has the pairs (1,5) and (2,4): answer 2.

✗ Buggy code. Find the bug before you read the solution.

```
static int countPairs(int[] a, int target) {
    int l = 0, r = a.length - 1, count = 0;
    while (l < r) {
        int sum = a[l] + a[r];
        if (sum == target) count++;
        else if (sum < target) l++;
        else r--;
    }
    return count;
}
```

Test: countPairs({1, 2, 3, 4, 5}, 6) → **Expected:** 2 **Got:** never finishes (infinite loop)

Solution 2.2 Count Pairs With a Given Sum

Why it fails. When sum == target the branch only increments count; neither pointer moves. The next loop pass sees exactly the same l and r, so the program counts the same pair for ever.

✓ Fixed code (changed lines are marked ✓)

```
static int countPairs(int[] a, int target) {
    int l = 0, r = a.length - 1, count = 0;
    while (l < r) {
        int sum = a[l] + a[r];
        if (sum == target) { count++; l++; r--; }   // ✓ move both pointers
        else if (sum < target) l++;
        else r--;
    }
    return count;
}
```

Remember: Check every branch of a two-pointer loop: each one must move at least one pointer, otherwise the program hangs.

Problem 2.3 Most Vowels in a Window

State not reset between windows · Medium

Task. Find the maximum number of vowels in any substring of length k. For "abciiidef" with k = 3 the best window is "iii", so the answer is 3.

✗ Buggy code. Find the bug before you read the solution.

```
static int maxVowels(String s, int k) {
    int best = 0, count = 0;
    for (int start = 0; start + k <= s.length(); start++) {
        for (int i = start; i < start + k; i++)
            if ("aeiou".indexOf(s.charAt(i)) >= 0) count++;
        best = Math.max(best, count);
    }
    return best;
}
```

Test: maxVowels("abciiidef", 3) → **Expected:** 3 **Got:** 12

Solution 2.3 Most Vowels in a Window

Why it fails. count is declared outside the loop, so it keeps growing from one window to the next. By the last window it holds the vowels of all windows added together (12), which is far more than any single window of 3 letters can contain.

✓ Fixed code (changed lines are marked ✓)

```
static int maxVowels(String s, int k) {
    int best = 0;
    for (int start = 0; start + k <= s.length(); start++) {
        int count = 0;   // ✓ fresh counter for every window
        for (int i = start; i < start + k; i++)
            if ("aeiou".indexOf(s.charAt(i)) >= 0) count++;
        best = Math.max(best, count);
    }
    return best;
}
```

Remember: Variables that describe one window or one pass must be reset at the start of that window or pass. Declare them inside the loop.

3 PART 3 Binary Search

Binary search is simple but unforgiving: one wrong +1, one wrong <= or one unsorted input breaks it.

Bug	How to spot it	Fix pattern
low = mid / high = mid (no ±1)	Infinite loop when the window narrows to 2	Always low = mid + 1 or high = mid - 1, never bare mid
< vs <= in the loop condition	Misses the last candidate, or checks one too many	Use while (low <= high) when both are valid candidates
Mid calculation overflow	Wrong answer only on very large inputs	Compute low + (high - low) / 2
Search on unsorted/wrong-direction data	Wrong answer, no runtime error	Confirm the array is sorted the way the logic assumes
Inconsistent 'not found' return	Sometimes -1, sometimes the wrong index	Decide what low/high mean at exit; return from one place

Problem 3.1 Binary Search: Missing the Last Element

< vs <= in the loop condition · Easy

Task. Return the index of target in a sorted array, or -1. In {2, 5, 8, 12, 16} the value 16 is at index 4.

✗ Buggy code. Find the bug before you read the solution.

```
static int search(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Test: search(a, 16) and search(a, 2) → **Expected:** `4 0` **Got:** `-1 0`

Solution 3.1 Binary Search: Missing the Last Element

Why it fails. Searching for 16: lo = 0, hi = 4, then lo = 3, then lo = 4. Now lo == hi == 4, the loop condition lo < hi is false, and the loop exits without ever checking index 4. When both pointers meet, one candidate is still left to test.

✓ Fixed code (changed lines are marked ✓)

```
static int search(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {   // ✓ lo == hi is still one candidate
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Remember: If lo and hi are both valid candidate positions, loop while (lo <= hi). Test the target at both ends of the array, not just the middle.

Problem 3.2 Search in a Descending Array

Search on unsorted / wrong-direction data · Medium

Task. The marks are stored from highest to lowest: {90, 70, 50, 30, 10}. Return the index of the target (30 is at index 3).

✗ Buggy code. Find the bug before you read the solution.

```
static int searchDesc(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Test: searchDesc({90, 70, 50, 30, 10}, 30) → **Expected:** `3` **Got:** `-1`

Solution 3.2 Search in a Descending Array

Why it fails. This is the standard ascending binary search. At mid = 2 the value is 50, which is bigger than 30, and the code thinks 'too big, go left'. But in a descending array the smaller values are on the right, so the search runs away from 30. There is no error message, just a wrong answer.

✔ Fixed code (changed lines are marked ✔)

```
static int searchDesc(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[mid] > t) lo = mid + 1;   // ✔ descending: bigger means go right
        else hi = mid - 1;
    }
    return -1;
}
```

Remember: Binary search only works if the data is sorted in the direction your comparisons assume. Check the order first, then write the comparison.

Problem 3.3 First Occurrence of a Value

Inconsistent exit / wrong index returned · Medium

Task. The sorted array may contain duplicates. Return the index of the FIRST occurrence of the target, or -1. In {1, 2, 2, 2, 2, 3} the first 2 is at index 1.

✗ Buggy code. Find the bug before you read the solution.

```
static int firstIndex(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        else if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Test: firstIndex({1, 2, 2, 2, 2, 3}, 2) → **Expected:** 1 **Got:** 2

Solution 3.3 First Occurrence of a Value

Why it fails. The code returns as soon as it sees any 2. The first midpoint is index 2, which holds a 2, so it answers 2, but the first 2 is at index 1. 'Found one' is not the same as 'found the first one'.

✔ Fixed code (changed lines are marked ✔)

```
static int firstIndex(int[] a, int t) {
    int lo = 0, hi = a.length - 1, ans = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) { ans = mid; hi = mid - 1; }   // ✔ remember it, keep looking left
        else if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return ans;   // ✔ single exit point
}
```

Remember: Decide what the search is looking for. For first/last occurrence, store the answer when you find it, keep narrowing towards the side you want, and return from one place at the end.

4 PART 4 Sorting

A sort is only as correct as its comparator. State what should come first, and say what happens on ties.

Bug	How to spot it	Fix pattern
Unstable sort where stability matters	Equal-key elements' order changes unexpectedly	Use a stable sort, or add an explicit tiebreaker
Inverted comparator	Sorts descending when ascending was wanted	Comparator returns negative when a belongs before b
Non-transitive comparator	Sort throws or gives inconsistent order	Ensure the ordering is strict and transitive
Partition off-by-one (quicksort)	Infinite recursion/stack overflow on sorted input	Re-derive the boundary by hand; use 3-way partition
Comparator returns 0 incorrectly	Different elements silently treated as equal	Return 0 only when every field that matters is tied

Problem 4.1 Rank Students With Ties

Missing tiebreaker (stability) · Medium

Task. Rank students by marks, highest first. Students with equal marks must appear alphabetically. Expected: Asha, Bala (90), then Meena, Ravi (80).

Given helper code (correct)

```
static class S {
    String name; int marks;
    S(String n, int m) { name = n; marks = m; }
}
```

✗ **Buggy code. Find the bug before you read the solution.**

```
static List<String> rank(List<S> list) {
    list.sort((x, y) -> Integer.compare(y.marks, x.marks));
    List<String> out = new ArrayList<>();
    for (S s : list) out.add(s.name);
    return out;
}
```

Test: rank([Ravi 80, Asha 90, Meena 80, Bala 90]) → **Expected:** `[Asha, Bala, Meena, Ravi]` **Got:** `[Asha, Bala, Ravi, Meena]`

Solution 4.1 Rank Students With Ties

Why it fails. The comparator looks only at marks. For equal marks it returns 0, so the sort keeps the original input order (Ravi before Meena). That order is an accident of the input, not a rule you wrote.

✓ **Fixed code (changed lines are marked ✓)**

```
static List<String> rank(List<S> list) {
    list.sort((x, y) -> {
        int c = Integer.compare(y.marks, x.marks);
        return c != 0 ? c : x.name.compareTo(y.name);   // ✓ explicit tiebreaker
    });
    List<String> out = new ArrayList<>();
    for (S s : list) out.add(s.name);
    return out;
}
```

Remember: If ties matter, never leave them to chance: add an explicit tiebreaker as the second rule of the comparator.

Problem 4.2 Sorting Large Numbers

Broken comparator (overflow) · Medium

Task. Sort an Integer array in ascending order. {1500000000, -1500000000} must become {-1500000000, 1500000000}.

✗ Buggy code. Find the bug before you read the solution.

```
static Integer[] sortAsc(Integer[] a) {
    Arrays.sort(a, (x, y) -> x - y);
    return a;
}
```

Test: sortAsc({1500000000, -1500000000}) → **Expected:** `[-1500000000, 1500000000]`
Got: `[1500000000, -1500000000]`

Solution 4.2 Sorting Large Numbers

Why it fails. x - y looks like a neat shortcut, but 1500000000 - (-1500000000) = 3000000000 does not fit in an int. It overflows to a negative number, so the comparator claims that the larger value is the smaller one.

✔ Fixed code (changed lines are marked ✔)

```
static Integer[] sortAsc(Integer[] a) {
    Arrays.sort(a, (x, y) -> Integer.compare(x, y));   // ✔ never subtract
    return a;
}
```

Remember: A comparator must tell the truth for ALL inputs. Use Integer.compare(x, y) instead of x - y.

Problem 4.3 Quicksort on Sorted Input

Partition off-by-one · Hard

Task. Sort an array with quicksort. It must work for any input, including an already sorted array such as {1, 2, 3, 4, 5}.

Given helper code (correct)

```
static int partition(int[] a, int lo, int hi) {   // Lomuto partition
(correct)
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++)
        if (a[j] < pivot) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; }
    int t = a[i]; a[i] = a[hi]; a[hi] = t;
    return i;   // final position of the pivot
}
```

✗ Buggy code. Find the bug before you read the solution.

```
static void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p);
    quickSort(a, p + 1, hi);
}
```

Test: quickSort({1, 2, 3, 4, 5}) → **Expected:** `[1, 2, 3, 4, 5]` **Got:** `Exception: StackOverflowError`

Solution 4.3 Quicksort on Sorted Input

Why it fails. partition puts the pivot at its final index p, so the left part is lo..p-1. The buggy call includes p itself. On sorted input the pivot (5) is the maximum, so p == hi and quickSort(a, lo, p) is exactly the same call as before: infinite recursion.

✔ Fixed code (changed lines are marked ✔)

```
static void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);   // ✔ pivot is already in place
    quickSort(a, p + 1, hi);
}
```

Remember: After partitioning, the pivot is finished. Recurse on lo..p-1 and p+1..hi, never on a range that still contains p.

5 PART 5 Linked Lists

Linked-list bugs are pointer bugs. Save a pointer before you overwrite it, and handle empty and one-node lists.

Bug	How to spot it	Fix pattern
Overwriting next before saving it	Reversing/rotating truncates the list	Save next = curr.next as the first line, before reassigning
Fast/slow pointer off-by-one	Middle node or cycle detection is off by one	Trace a 2-node and 3-node list by hand
Missing null/single-node check	Crashes on an empty or single-node list	Explicitly handle head == null and head.next == null
Dangling prev/tail pointer	New list is missing a link or ends early	Re-point every affected pointer after each change
Manual memory leak (C / C++)	Memory keeps growing despite 'deleted' nodes	free()/delete only after saving the pointer you still need

Helper code used by the problems in this part (correct)

```
static class Node {
    int val; Node next;
    Node(int v) { val = v; }
}
static Node build(int... v) {
    Node d = new Node(0), c = d;
    for (int x : v) { c.next = new Node(x); c = c.next; }
    return d.next;
}
static String str(Node h) {
    StringBuilder sb = new StringBuilder();
    for (; h != null; h = h.next) sb.append(h.val).append(h.next != null ? " -> "
: "");
    return sb.toString();
}
```

Problem 5.1 Middle of a Linked List

Fast/slow pointer off-by-one · Medium

Task. Return the middle value. For an even-length list return the SECOND middle: 1→2→3→4 gives 3, and 1→2→3→4→5 gives 3.

Uses the helper code shown at the start of Part 5.

✗ Buggy code. Find the bug before you read the solution.

```
static int middle(Node head) {
    Node slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow.val;
}
```

Test: middle(1-2-3-4) and middle(1-2-3-4-5) → **Expected:** `3 3` **Got:** `2 3`

Solution 5.1 Middle of a Linked List

Why it fails. The buggy loop stops one step early when fast has one more node ahead. For 1-2-3-4, fast stops at node 3, so slow is still at node 2 (the first middle). Odd lists work by luck; even lists reveal the off-by-one.

✓ Fixed code (changed lines are marked ✓)

```
static int middle(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {   // ✓ stop only when fast runs out
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow.val;
}
```

Remember: Trace the smallest lists by hand: 2 nodes and 3 nodes. The loop condition fast != null && fast.next != null returns the second middle.

Problem 5.2 Delete All Nodes With a Value

Dangling pointer / head special case · Hard

Task. Delete every node whose value equals val and return the new head. Deleting 2 from 2→2→3 must give 3.

Uses the helper code shown at the start of Part 5.

✗ Buggy code. Find the bug before you read the solution.

```
static Node deleteAll(Node head, int val) {
    Node cur = head;
    while (cur != null && cur.next != null) {
        if (cur.next.val == val) cur.next = cur.next.next;
        cur = cur.next;
    }
    return head;
}
```

Test: deleteAll(2-2-3, 2) → **Expected:** `3` **Got:** `2 -> 3`

Solution 5.2 Delete All Nodes With a Value

Why it fails. Three mistakes in one: (1) the head node is never checked, because the loop only looks at cur.next; (2) after a deletion cur moves forward and skips the next node, which may also need deleting; (3) the old head is returned even if it should be gone.

✔ Fixed code (changed lines are marked ✔)

```
static Node deleteAll(Node head, int val) {
    Node dummy = new Node(0);   // ✔ dummy node in front of head
    dummy.next = head;
    Node cur = dummy;
    while (cur.next != null) {
        if (cur.next.val == val) cur.next = cur.next.next;
        else cur = cur.next;   // ✔ move only when nothing was deleted
    }
    return dummy.next;
}
```

Remember: Use a dummy node before the head so that deleting the first node is the same as deleting any other. Advance cur only when you did NOT delete.

Problem 5.3 Adding at the Tail

Dangling tail pointer · Medium

Task. Keep head and tail pointers so that add(v) appends in O(1). After add(1), add(2), add(3) the list must read 1→2→3.

Uses the helper code shown at the start of Part 5.

✗ Buggy code. Find the bug before you read the solution.

```
static Node head, tail;
static void add(int v) {
    Node n = new Node(v);
    if (head == null) { head = n; tail = n; }
    else { tail.next = n; }
}
```

Test: add(1); add(2); add(3) → **Expected:** `1 -> 2 -> 3` **Got:** `1 -> 3`

Solution 5.3 Adding at the Tail

Why it fails. After linking the new node, tail still points at node 1. So add(3) attaches node 3 to node 1 and overwrites the link to node 2: the list silently loses a node.

✔ Fixed code (changed lines are marked ✔)

```
static Node head, tail;
static void add(int v) {
    Node n = new Node(v);
    if (head == null) { head = n; tail = n; }
    else { tail.next = n; tail = n; }   // ✔ move tail to the new node
}
```

Remember: Every time you change the list, re-point EVERY affected pointer (head, tail, prev, next). Ask after each operation: where does tail point now?

6 PART 6 Stacks & Queues

Stacks are LIFO and queues are FIFO. Most bugs are the wrong end, the wrong order or an empty structure.

Bug	How to spot it	Fix pattern
Pop/peek without an empty check	Crash or undefined behavior when empty	Check isEmpty() before every pop()/peek()
Queue-via-stacks transfer bug	Wrong order, or every call costs O(n)	Transfer only when the 'out' stack is empty
Circular queue wraparound	Index runs off the end of the array	Use index = (index + 1) % capacity
LIFO/FIFO logic swapped	Output order is exactly reversed	Re-confirm which end is pushed/popped from
Returning the whole structure, not its top	Type errors, or wrong value used downstream	Return the accessor (top/front), never the structure

Problem 6.1 Reverse a String

LIFO / FIFO logic swapped · Easy

Task. Reverse a string using a data structure. "stack" must become "kcats".

✗ **Buggy code. Find the bug before you read the solution.**

```
static String reverse(String s) {
    Queue<Character> q = new LinkedList<>();
    for (char c : s.toCharArray()) q.add(c);
    StringBuilder sb = new StringBuilder();
    while (!q.isEmpty()) sb.append(q.remove());
    return sb.toString();
}
```

Test: reverse("stack") → **Expected:** `kcats` **Got:** `stack`

Solution 6.1 Reverse a String

Why it fails. A queue is FIFO: first in, first out. Characters come out in the same order they went in, so the string is unchanged. Reversing needs LIFO (last in, first out), which is a stack.

✓ **Fixed code (changed lines are marked ✓)**

```
static String reverse(String s) {
    Deque<Character> stack = new ArrayDeque<>();   // ✓ stack = LIFO
    for (char c : s.toCharArray()) stack.push(c);
    StringBuilder sb = new StringBuilder();
    while (!stack.isEmpty()) sb.append(stack.pop());
    return sb.toString();
}
```

Remember: Before choosing a structure, confirm which end is pushed and which is popped. Reverse order = stack. Same order = queue.

Problem 6.2 Queue Using Two Stacks

Queue-via-stacks transfer bug | Hard

Task. Build a FIFO queue from two stacks. After push(1), push(2), pop, push(3), pop, pop the answers must be 1, 2, 3.

✗ **Buggy code. Find the bug before you read the solution.**

```
static Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
static void push(int x) { in.push(x); }
static int pop() {
    while (!in.isEmpty()) out.push(in.pop());
    return out.pop();
}
```

Test: push 1, 2; pop; push 3; pop; pop → **Expected:** `1 2 3` **Got:** `1 3 2`

Solution 6.2 Queue Using Two Stacks

Why it fails. After the first pop, element 2 is waiting in the 'out' stack. The buggy pop moves the new element 3 on top of it, so 3 comes out before 2. Transferring while out still holds older elements reverses the order of the queue.

✔ **Fixed code (changed lines are marked ✔)**

```
static Deque<Integer> in = new ArrayDeque<>(), out = new ArrayDeque<>();
static void push(int x) { in.push(x); }
static int pop() {
    if (out.isEmpty())   // ✔ transfer only when out is empty
        while (!in.isEmpty()) out.push(in.pop());
    return out.pop();
}
```

Remember: Move elements from 'in' to 'out' ONLY when 'out' is empty. Then 'out' always holds the oldest elements, in the correct order.

Problem 6.3 Evaluate a Postfix Expression

Pop order (LIFO) mistake | Medium

Task. Evaluate an expression written in postfix form. "5 3 -" means 5 - 3 = 2 and "8 2 /" means 8 / 2 = 4.

✗ **Buggy code. Find the bug before you read the solution.**

```
static int evalPostfix(String expr) {
    Deque<Integer> st = new ArrayDeque<>();
    for (String t : expr.split(" ")) {
        if (t.matches("\\d+")) { st.push(Integer.parseInt(t)); continue; }
        int a = st.pop(), b = st.pop();
        switch (t) {
            case "+": st.push(a + b); break;
            case "-": st.push(a - b); break;
            case "*": st.push(a * b); break;
            default:  st.push(a / b);
        }
    }
    return st.pop();
}
```

Test: evalPostfix("5 3 -") and evalPostfix("8 2 /") → **Expected:** `2 4` **Got:** `-2 0`

Solution 6.3 Evaluate a Postfix Expression

Why it fails. Stack order is last-in, first-out: for "5 3 -" the first pop returns 3 (the RIGHT operand) and the second pop returns 5. The buggy code computed a - b = 3 - 5. Addition and multiplication hide this bug because they do not care about order.

✔ Fixed code (changed lines are marked ✔)

```
static int evalPostfix(String expr) {
    Deque<Integer> st = new ArrayDeque<>();
    for (String t : expr.split(" ")) {
        if (t.matches("\\d+")) { st.push(Integer.parseInt(t)); continue; }
        int a = st.pop(), b = st.pop();   // a = RIGHT operand, b = LEFT operand
        switch (t) {
            case "+": st.push(b + a); break;
            case "-": st.push(b - a); break;   // ✔ left operand first
            case "*": st.push(b * a); break;
            default:  st.push(b / a);   // ✔
        }
    }
    return st.pop();
}
```

Remember: The first value popped is the right operand. Always compute (second pop) operator (first pop). Test with - and / to catch this.

7 PART 7 Trees & Binary Search Trees

Every tree function starts with the null case and then trusts recursion. Know which of pre-, in- or post-order you need.

Bug	How to spot it	Fix pattern
Missing null-root base case	Crashes the instant recursion reaches a leaf's child	Every recursive function's first line: handle node == null
BST comparison direction reversed	Search/insert goes into the wrong subtree	Smaller values go left, larger go right
Traversal order confusion	Values are correct but in the wrong order	Pre-order before recursing; in-order between; post-order after
BST deletion, two-children case	Tree silently violates the BST property	Replace with the in-order successor, then delete that node
Off-by-one in height/depth	Reported height is one more or less than expected	Decide once if a leaf has height 0 or 1; apply it everywhere

Helper code used by the problems in this part (correct)

```
static class N {
    int v; N l, r;
    N(int v) { this.v = v; }
}
static N insert(N root, int x) {
    if (root == null) return new N(x);
    if (x < root.v) root.l = insert(root.l, x);
    else root.r = insert(root.r, x);
    return root;
}
static N build(int... xs) {
    N root = null;
    for (int x : xs) root = insert(root, x);
    return root;
}
```

Problem 7.1 Delete a Node With Two Children

BST deletion, two-children case — Hard

Task. Delete a key from a BST. Build the tree 50, 30, 70, 20, 40, 60, 80 and delete 50 (the root). An in-order print of the result must stay sorted.

Uses the helper code shown at the start of Part 7.

Given helper code (correct)

```
static void collect(N n, List<Integer> out) {   // in-order walk (correct)
    if (n == null) return;
    collect(n.l, out); out.add(n.v); collect(n.r, out);
}
static String show(N root) { List<Integer> o = new ArrayList<>(); collect(root, o);
return o.toString(); }
```

✗ **Buggy code. Find the bug before you read the solution.**

```
static N delete(N root, int k) {
    if (root == null) return null;
    if (k < root.v) root.l = delete(root.l, k);
    else if (k > root.v) root.r = delete(root.r, k);
    else {
        if (root.l == null) return root.r;
        if (root.r == null) return root.l;
        root.v = root.l.v;
        root.l = delete(root.l, root.l.v);
    }
    return root;
}
```

Test: delete(50) from the tree, then print in-order → **Expected:** `[20, 30, 40, 60, 70, 80]`
Got: `[20, 40, 30, 60, 70, 80]`

Solution 7.1 Delete a Node With Two Children

Why it fails. The replacement value must be larger than everything in the left subtree and smaller than everything in the right subtree. The buggy code copies the left child (30), which is not the largest on the left, so 40 ends up on the wrong side of 30 and the BST property is broken (the in-order list is no longer sorted). No exception is thrown.

✔ **Fixed code (changed lines are marked ✔)**

```
static N delete(N root, int k) {
    if (root == null) return null;
    if (k < root.v) root.l = delete(root.l, k);
    else if (k > root.v) root.r = delete(root.r, k);
    else {
        if (root.l == null) return root.r;
        if (root.r == null) return root.l;
        N s = root.r;                       // ✔ find the in-order successor:
        while (s.l != null) s = s.l;        // ✔ smallest value in the right subtree
        root.v = s.v;
        root.r = delete(root.r, s.v);       // ✔ delete the successor from the right
    }
    return root;
}
```

Remember: Two children: replace with the in-order successor (smallest node of the right subtree), then delete that successor from the right subtree.

Problem 7.2 Print a BST in Sorted Order

Traversal order confusion · Easy

Task. Collect the values of a BST in ascending order. For the tree built from 50, 30, 70, 20, 40, 60, 80 the answer is 20, 30, 40, 50, 60, 70, 80.

Uses the helper code shown at the start of Part 7.

✗ Buggy code. Find the bug before you read the solution.

```
static void sortedValues(N n, List<Integer> out) {
    if (n == null) return;
    out.add(n.v);
    sortedValues(n.l, out);
    sortedValues(n.r, out);
}
```

Test: sortedValues(tree) → **Expected:** `[20, 30, 40, 50, 60, 70, 80]` **Got:** `[50, 30, 20, 40, 70, 60, 80]`

Solution 7.2 Print a BST in Sorted Order

Why it fails. The node is added before its children, which is pre-order. All the values are present but in the wrong order. Sorted output from a BST needs in-order: left subtree, then node, then right subtree.

✓ Fixed code (changed lines are marked ✓)

```
static void sortedValues(N n, List<Integer> out) {
    if (n == null) return;
    sortedValues(n.l, out);
    out.add(n.v);   // ✓ visit the node BETWEEN the two recursive calls
    sortedValues(n.r, out);
}
```

Remember: Pre-order = node first. In-order = node between. Post-order = node last. For a BST, in-order gives sorted order.

Problem 7.3 Height of a Binary Tree

Off-by-one in height / depth · Medium

Task. Height = number of EDGES on the longest path from the root down to a leaf. A single node has height 0; the 7-node tree 50, 30, 70, 20, 40, 60, 80 has height 2.

Uses the helper code shown at the start of Part 7.

✗ Buggy code. Find the bug before you read the solution.

```
static int height(N n) {
    if (n == null) return 0;
    return 1 + Math.max(height(n.l), height(n.r));
}
```

Test: height(7-node tree) and height(single node) → **Expected:** `2 0` **Got:** `3 1`

Solution 7.3 Height of a Binary Tree

Why it fails. Returning 0 for an empty tree makes a leaf count as 1, so this code actually counts NODES on the longest path, not edges. Everything is shifted up by exactly one.

✓ Fixed code (changed lines are marked ✓)

```
static int height(N n) {
    if (n == null) return -1;   // ✓ empty tree = -1 so a leaf = 0
    return 1 + Math.max(height(n.l), height(n.r));
}
```

Remember: Decide once: a leaf has height 0 (edges) or 1 (nodes). Then the empty tree must be -1 or 0 to match. Check it on a single node.

8 PART 8 Graphs (BFS/DFS)

In graphs the visited set is everything: when you mark it, what it covers and whether it is fresh for each query.

Bug	How to spot it	Fix pattern
Marking visited on dequeue, not enqueue (BFS)	Same node processed multiple times	Mark visited when added to the queue, not when popped
Forgetting to mark the start node visited	Traversal revisits or duplicates the start	Mark the start visited before the main loop begins
Directed edge traversed backwards	Reaches nodes it shouldn't, or misses some	Confirm the adjacency list's direction matches the edges
DFS stack overflow on deep graphs	Works on small tests, crashes on deep ones	Convert to iterative DFS with an explicit stack
Visited state reused across calls	A second query returns nothing or the wrong result	Use a fresh visited set per call

Problem 8.1 BFS Order

Forgetting to mark the start node visited · Easy

Task. Return the nodes in BFS order starting from node 0 in the undirected graph with edges 0-1 and 0-2. Every node must appear once: [0, 1, 2].

Given helper code (correct)

```
static List<List<Integer>> graph(int n, int[][] edges, boolean directed) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);
        if (!directed) g.get(e[1]).add(e[0]);
    }
    return g;
}
```

✗ **Buggy code. Find the bug before you read the solution.**

```
static List<Integer> bfs(List<List<Integer>> g, int s) {
    boolean[] visited = new boolean[g.size()];
    Queue<Integer> q = new LinkedList<>();
    List<Integer> order = new ArrayList<>();
    q.add(s);
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : g.get(u))
            if (!visited[v]) { visited[v] = true; q.add(v); }
    }
    return order;
}
```

Test: BFS from 0 on edges 0-1, 0-2 → **Expected:** `[0, 1, 2]` **Got:** `[0, 1, 2, 0]`

Solution 8.1 BFS Order

Why it fails. Neighbours get marked when they are added, but the start node is never marked. When node 1 looks at its neighbour 0, 0 still looks unvisited, so it is queued a second time and appears twice in the output.

✔ Fixed code (changed lines are marked ✔)

```
static List<Integer> bfs(List<List<Integer>> g, int s) {
    boolean[] visited = new boolean[g.size()];
    Queue<Integer> q = new LinkedList<>();
    List<Integer> order = new ArrayList<>();
    q.add(s);
    visited[s] = true;   // ✔ mark the start before the loop
    while (!q.isEmpty()) {
        int u = q.poll();
        order.add(u);
        for (int v : g.get(u))
            if (!visited[v]) { visited[v] = true; q.add(v); }
    }
    return order;
}
```

Remember: Mark a node visited at the moment you add it to the queue, and that includes the start node.

Problem 8.2 Nodes Reachable in a Directed Graph

Directed edge traversed backwards · Medium

Task. Directed edges: 0→1, 1→2, 2→3. Count the nodes reachable from node 2, including 2 itself. Only 2 and 3 can be reached, so the answer is 2.

Given helper code (correct)

```
static int dfs(List<List<Integer>> g, int u, boolean[] seen) {
    seen[u] = true;
    int count = 1;
    for (int v : g.get(u))
        if (!seen[v]) count += dfs(g, v, seen);
    return count;
}
```

✗ Buggy code. Find the bug before you read the solution.

```
static int countReachable(int n, int[][] edges, int start) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);
        g.get(e[1]).add(e[0]);
    }
    return dfs(g, start, new boolean[n]);
}
```

Test: countReachable(4 nodes, 0→1, 1→2, 2→3, start 2) → **Expected:** 2 **Got:** 4

Solution 8.2 Nodes Reachable in a Directed Graph

Why it fails. The second add line stores every edge in both directions, which is correct for an undirected graph. For a directed graph it lets the DFS walk 2 → 1 → 0 against the arrows, so all 4 nodes look reachable.

✔ Fixed code (changed lines are marked ✔)

```
static int countReachable(int n, int[][] edges, int start) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    for (int[] e : edges) {
        g.get(e[0]).add(e[1]);   // ✔ directed: only from -> to (reverse line removed)
    }
    return dfs(g, start, new boolean[n]);
}
```

Remember: Build the adjacency list to match the problem: directed edge a→b is stored once (a's list). Undirected edges are stored twice.

Problem 8.3 DFS on a Very Long Chain

DFS stack overflow on deep graphs · Hard

Task. A chain of 200000 nodes (0→1→2→...→199999). Count the nodes reachable from node 0. The answer is 200000.

✗ Buggy code. Find the bug before you read the solution.

```
static int countNodes(List<List<Integer>> g, int u, boolean[] seen) {
    seen[u] = true;
    int count = 1;
    for (int v : g.get(u))
        if (!seen[v]) count += countNodes(g, v, seen);
    return count;
}
```

Test: countNodes on a 200000-node chain → **Expected:** `200000` **Got:** `Exception: StackOverflowError`

Solution 8.3 DFS on a Very Long Chain

Why it fails. Each recursive call uses a stack frame, and the chain makes the recursion 200000 calls deep. The JVM's call stack holds far fewer frames, so the program crashes, although it works perfectly on small test graphs.

✓ Fixed code (changed lines are marked ✓)

```
static int countNodes(List<List<Integer>> g, int s, boolean[] seen) {
    Deque<Integer> stack = new ArrayDeque<>();   // ✓ explicit stack, no recursion
    stack.push(s);
    seen[s] = true;
    int count = 0;
    while (!stack.isEmpty()) {
        int u = stack.pop();
        count++;
        for (int v : g.get(u))
            if (!seen[v]) { seen[v] = true; stack.push(v); }
    }
    return count;
}
```

Remember: If the graph can be very deep, convert recursive DFS into iterative DFS with your own Deque. Your stack lives on the heap and can be as large as the data.

9 PART 9 Dynamic Programming

DP needs a clear meaning for each cell, correct base cases, the right min or max, and one consistent indexing scheme.

Bug	How to spot it	Fix pattern
Wrong base case value	Every value is offset by a constant amount	Match the base case to the literal empty/zero case
0/1 knapsack iterating capacity forwards	Items appear reusable even though each is used once	Iterate capacity HIGH to LOW for 0/1 knapsack
Inverted comparison for min/max	The answer is the worst case, not the best	Max updates on <; min updates on >
Recursion with no memoization	Correct but exponential; times out on large inputs	Confirm every computed value is stored before it's returned
1-indexed vs 0-indexed drift	Off-by-one errors on specific input sizes	Pick one convention; write down what dp[0] means

Problem 9.1 Climbing Stairs

Wrong base case value · Easy

Task. You can climb 1 or 2 steps at a time. How many different ways reach step n? ways(5) must be 8 (sequence 1, 1, 2, 3, 5, 8).

✗ Buggy code. Find the bug before you read the solution.

```
static int ways(int n) {
    int[] dp = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

Test: ways(5) → **Expected:** 8 **Got:** 5

Solution 9.1 Climbing Stairs

Why it fails. The recurrence is correct, but dp[0] = 0 says there is no way to be at step 0. Every later value is built from the base values, so the whole table is shifted: 0, 1, 1, 2, 3, 5 instead of 1, 1, 2, 3, 5, 8.

✓ Fixed code (changed lines are marked ✓)

```
static int ways(int n) {
    int[] dp = new int[n + 1];
    dp[0] = 1;   // ✓ one way to stand at the start: do nothing
    dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}
```

Remember: A whole table that is off by a constant pattern points to the base case. Ask: how many ways are there to do nothing? Exactly one.

Problem 9.2 Fibonacci That Never Finishes

Recursion with no memoization · Medium

Task. Compute fib(50) = 12586269025. The program must answer instantly.

✗ Buggy code. Find the bug before you read the solution.

```
static long fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Test: fib(50) → **Expected:** 12586269025 **Got:** never finishes (infinite loop)

Solution 9.2 Fibonacci That Never Finishes

Why it fails. The code is correct but fib(48) and fib(47) are computed again and again from scratch: the number of calls roughly doubles with every +1 in n. For n = 50 that is tens of billions of calls (minutes or hours), so the program appears frozen.

✓ Fixed code (changed lines are marked ✓)

```
static long[] memo = new long[100];
static long fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];            // ✓ reuse stored answer
    return memo[n] = fib(n - 1) + fib(n - 2);    // ✓ store before returning
}
```

Remember: If a recursive function recomputes the same arguments, store each answer in a memo array the first time and look it up afterwards ($O(2^n)$ becomes $O(n)$).

Problem 9.3 Longest Common Subsequence

1-indexed vs 0-indexed drift · Hard

Task. dp[i][j] = LCS length of the first i characters of a and the first j characters of b. For "abcde" and "ace" the answer is 3 ("ace").

✗ **Buggy code. Find the bug before you read the solution.**

```
static int lcs(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (a.charAt(i) == b.charAt(j)) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[m][n];
}
```

Test: lcs("abcde", "ace") → **Expected:** `3` **Got:** `Exception: StringIndexOutOfBoundsException`

Solution 9.3 Longest Common Subsequence

Why it fails. The table is 1-indexed (row i means 'first i characters') but Strings are 0-indexed. The i-th character of a string is charAt(i - 1). When i reaches m, charAt(m) is outside the string.

✓ **Fixed code (changed lines are marked ✓)**

```
static int lcs(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1] + 1;
// ✓ dp is 1-based, strings 0-based
            else dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[m][n];
}
```

Remember: Write down what dp[0] means and which index convention each array uses. Table index i corresponds to string index i - 1.

10 PART 10 Recursion & Backtracking

Backtracking is choose, explore, un-choose. Save copies, stop early and keep the recursion index right.

Bug	How to spot it	Fix pattern
Missing or wrong base case	Stack overflow or infinite recursion	Every path must hit a base case; trace the smallest input
Forgetting to backtrack (un-choose)	Later branches see leftover state	Undo exactly what was done after the recursive call returns
Storing a reference instead of a snapshot	All results end up identical, often empty	Copy the current path when you store it
No pruning	Correct answer, but times out on larger inputs	Cut a branch the moment it can't lead to a valid answer
Off-by-one in the next recursive index	An element is skipped, or reused when it shouldn't be	Next call starts at i (reuse) or i + 1 (no reuse)

Problem 10.1 Generate Permutations

Storing a reference instead of a snapshot · Medium

Task. List all permutations of {1, 2, 3}. There are 6, and each one must be a complete list like [1, 2, 3].

✗ Buggy code. Find the bug before you read the solution.

```
static void permute(int[] a, boolean[] used, List<Integer> path, List<List<Integer>> res) {
    if (path.size() == a.length) { res.add(path); return; }
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true; path.add(a[i]);
        permute(a, used, path, res);
        path.remove(path.size() - 1); used[i] = false;
    }
}
```

Test: permute({1, 2, 3}) → **Expected:** `[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]` **Got:** `[[], [], [], [], [], []]`

Solution 10.1 Generate Permutations

Why it fails. res.add(path) stores the SAME list object six times, not six separate lists. Backtracking keeps changing that one list, and when the recursion finishes it is empty again, so every stored result is [].

✓ Fixed code (changed lines are marked ✓)

```
static void permute(int[] a, boolean[] used, List<Integer> path, List<List<Integer>> res) {
    if (path.size() == a.length) { res.add(new ArrayList<>(path)); return; }   // ✓ store a copy
    for (int i = 0; i < a.length; i++) {
        if (used[i]) continue;
        used[i] = true; path.add(a[i]);
        permute(a, used, path, res);
        path.remove(path.size() - 1); used[i] = false;
    }
}
```

Remember: When you save a result inside backtracking, save a copy: new ArrayList<>(path).

Problem 10.2 Recursive Palindrome Check

Missing / wrong base case · Medium

Task. Check whether s[l..r] is a palindrome by comparing the outer characters and recursing inward. "racecar" and "abba" must both give true.

✗ Buggy code. Find the bug before you read the solution.

```
static boolean isPal(String s, int l, int r) {
    if (l == r) return true;
    if (s.charAt(l) != s.charAt(r)) return false;
    return isPal(s, l + 1, r - 1);
}
```

Test: isPal("racecar") and isPal("abba") → **Expected:** `true true` **Got:** `Exception: StringIndexOutOfBoundsException`

Solution 10.2 Recursive Palindrome Check

Why it fails. For odd-length words the pointers meet (l == r) in the middle, so "racecar" works. For even-length words they cross instead: (1, 2) becomes (2, 1) and l == r is never true. The recursion keeps going until charAt reads outside the string.

✓ Fixed code (changed lines are marked ✓)

```
static boolean isPal(String s, int l, int r) {
    if (l >= r) return true;   // ✓ pointers can also cross (even length)
    if (s.charAt(l) != s.charAt(r)) return false;
    return isPal(s, l + 1, r - 1);
}
```

Remember: Base cases must cover every way a recursion can end. With two moving pointers: l == r (odd length) AND l > r (even length), so use l >= r.

Problem 10.3 Valid Parentheses Generator

No pruning Hard

Task. List all valid strings of n = 5 pairs of parentheses (there are 42). Both versions give the right answer; compare how many recursive calls each one makes.

Given helper code (correct)

```
static int calls = 0;
static boolean valid(String s) {
    int bal = 0;
    for (char c : s.toCharArray()) { bal += c == '(' ? 1 : -1; if (bal < 0) return false; }
    return bal == 0;
}
```

✗ **Buggy code. Find the bug before you read the solution.**

```
static void gen(int n, String cur, List<String> res) {
    calls++;
    if (cur.length() == 2 * n) { if (valid(cur)) res.add(cur); return; }
    gen(n, cur + "(", res);
    gen(n, cur + ")", res);
}
```

Test: generate all valid strings for n = 5 → **Expected:** `42 results, 196 calls` **Got:** `42 results, 2047 calls`

Solution 10.3 Valid Parentheses Generator

Why it fails. The buggy version tries every one of the 2^{10} = 1024 strings of (and) and throws the invalid ones away at the very end (2047 calls). It is correct, but for n = 15 it would take billions of steps. The fixed version stops a branch as soon as it can never become valid.

✓ **Fixed code (changed lines are marked ✓)**

```
static void gen(int n, int open, int close, String cur, List<String> res) {
    calls++;
    if (cur.length() == 2 * n) { res.add(cur); return; }
    if (open < n)     gen(n, open + 1, close, cur + "(", res);   // ✓ prune: too many '('
    if (close < open) gen(n, open, close + 1, cur + ")", res);   // ✓ prune: ')' without a match
}
```

Remember: Prune early: cut a branch the moment it cannot lead to a valid answer (here: more ')' than '(' so far, or more than n '(').

11 PART 11 Greedy Algorithms

A greedy algorithm needs a proof. Check the sort key, the tie rule and whether an early choice can hurt later.

Bug	How to spot it	Fix pattern
Greedy applied where DP is required	Passes the example, fails on a subtler case	If you can't prove the local choice is optimal, find a counterexample
Wrong sort key before the greedy pass	Answer is close to optimal, but not quite	For max non-overlapping intervals, sort by END time
Off-by-one in overlap check	Touching intervals wrongly treated as overlapping	Re-read whether touching counts as overlap; use < vs <=
Ties broken inconsistently	Different runs give different valid answers	Add an explicit, deterministic tiebreaker
Greedy choice locked in too early	A later, better option is never considered	Check if the problem allows reconsidering earlier choices

Problem 11.1 Fewest Coins

Greedy applied where DP is required · Hard

Task. With coins {4, 3, 1}, find the minimum number of coins that make amount 6. The best answer is 2 coins (3 + 3).

✗ Buggy code. Find the bug before you read the solution.

```
static int minCoins(int[] coins, int amount) {   // coins sorted high to low
    int count = 0;
    for (int c : coins) {
        count += amount / c;
        amount %= c;
    }
    return count;
}
```

Test: minCoins({4, 3, 1}, 6) → **Expected:** 2 **Got:** 3

Solution 11.1 Fewest Coins

Why it fails. Greedy takes the biggest coin first: 4, then 1, then 1 = 3 coins. It commits to 4 and can never go back, but the best answer 3 + 3 does not use 4 at all. The code even passes simple examples such as 8 = 4 + 4, which is why this bug survives testing.

✔ Fixed code (changed lines are marked ✔)

```
static int minCoins(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)          // ✔ try every coin for every amount
        for (int c : coins)
            if (c <= a && dp[a - c] != Integer.MAX_VALUE)
                dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount];
}
```

Remember: A greedy choice is safe only if you can prove it. Try to find a counterexample; if one exists (like this one), use DP.

Problem 11.2 Can You Reach the End?

Greedy choice locked in too early · Hard

Task. a[i] is the maximum jump length from index i. Can you reach the last index? For {2, 3, 0, 0, 4} the answer is true: jump 1 step to index 1, then 3 steps to index 4.

✗ Buggy code. Find the bug before you read the solution.

```
static boolean canJump(int[] a) {
    int i = 0;
    while (i < a.length - 1) {
        if (a[i] == 0) return false;
        i += a[i];
    }
    return true;
}
```

Test: canJump({2, 3, 0, 0, 4}) → **Expected:** `true` **Got:** `false`

Solution 11.2 Can You Reach the End?

Why it fails. The buggy code always jumps the maximum distance. From index 0 it lands on index 2 (value 0) and is stuck, never considering the shorter jump to index 1 that leads to the end. The decision was locked in too early.

✓ Fixed code (changed lines are marked ✓)

```
static boolean canJump(int[] a) {
    int furthest = 0;
    for (int i = 0; i < a.length; i++) {
        if (i > furthest) return false;
        furthest = Math.max(furthest, i + a[i]);   // ✓ keep ALL options open
    }
    return true;
}
```

Remember: Do not commit to one jump. Track the furthest index reachable so far, which keeps every earlier option available.

Problem 11.3 Fractional Knapsack

Wrong sort key before the greedy pass · Medium

Task. Items (value, weight): (60, 10), (100, 20), (120, 30). A bag holds weight 50 and items may be split. The best total value is 240.

✗ Buggy code. Find the bug before you read the solution.

```
static double bestValue(int[] v, int[] w, int cap) {
    Integer[] idx = {0, 1, 2};
    Arrays.sort(idx, (x, y) -> v[y] - v[x]);   // biggest value first
    double total = 0;
    for (int i : idx) {
        if (cap == 0) break;
        int take = Math.min(w[i], cap);
        total += (double) v[i] * take / w[i];
        cap -= take;
    }
    return total;
}
```

Test: bestValue(values {60,100,120}, weights {10,20,30}, capacity 50) → **Expected:** `240.0` **Got:** `220.0`

Solution 11.3 Fractional Knapsack

Why it fails. Sorting by value picks the 120-value item first, but it is heavy (30 kg) and uses up most of the bag. The right measure is value per unit weight: 6, 5 and 4. The greedy loop is correct; the SORT KEY was wrong, so the answer is close to optimal but not quite.

✓ Fixed code (changed lines are marked ✓)

```
static double bestValue(int[] v, int[] w, int cap) {
    Integer[] idx = {0, 1, 2};
    Arrays.sort(idx, (x, y) -> Double.compare((double) v[y] / w[y], (double) v[x] / w[x]));
// ✓ best value PER KG first
    double total = 0;
    for (int i : idx) {
        if (cap == 0) break;
        int take = Math.min(w[i], cap);
        total += (double) v[i] * take / w[i];
        cap -= take;
    }
    return total;
}
```

Remember: A greedy algorithm is only as good as its sort key. Ask: which single number makes one choice better than another? (Here value divided by weight.)

12 PART 12 Hashing & Hash Maps

A hash map is only as reliable as its keys and its existence checks. Seed the base case and never change a map while you walk it.

Bug	How to spot it	Fix pattern
Forgetting to seed the base case	Off-by-one in every prefix-sum-plus-map answer	Seed the map with the empty-prefix case before the loop
Falsy-value existence check	A valid key with value 0/""/false looks missing	Use an explicit has()/in check, never a truthy check
Mutating a map while iterating it	Skipped entries or a runtime error	Collect keys to change first, then apply in a second pass
Using mutable objects as keys	Equal keys treated as different (or vice versa)	Use a primitive or canonical string/tuple as the key
Ignoring collisions in a hand-built hash table	Silent data loss when keys share a bucket	Store a bucket (list) per slot, or use open addressing

Problem 12.1 Two Sum

Falsy / default value used as 'missing' — Medium

Task. Return the indices of the two numbers that add up to the target. For {2, 7, 11, 15} and target 9 the answer is [0, 1].

✗ **Buggy code. Find the bug before you read the solution.**

```
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> idx = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        int j = idx.getOrDefault(target - a[i], 0);
        if (j > 0) return new int[]{j, i};
        idx.put(a[i], i);
    }
    return new int[]{};
}
```

Test: twoSum({2, 7, 11, 15}, 9) → **Expected:** `[0, 1]` **Got:** `[]`

Solution 12.1 Two Sum

Why it fails. The code uses 0 to mean 'not found', but 0 is also a valid stored index. The value 2 is stored with index 0, and the test j > 0 treats it as missing. The answer is lost exactly when the first element is part of the pair.

✓ **Fixed code (changed lines are marked ✓)**

```
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> idx = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        if (idx.containsKey(target - a[i]))   // ✓ explicit existence check
            return new int[]{idx.get(target - a[i]), i};
        idx.put(a[i], i);
    }
    return new int[]{};
}
```

Remember: Never use a normal value (0, "", false) to mean 'absent'. Test existence with containsKey (or compare with null).

Problem 12.2 Counting Points

Using mutable objects as keys · Medium

Task. Count how many times each (x, y) point appears. The points {1,2}, {3,4}, {1,2} contain 2 distinct points, and the most repeated one appears 2 times.

✗ **Buggy code. Find the bug before you read the solution.**

```
static String countPoints(int[][] pts) {
    Map<int[], Integer> counts = new HashMap<>();
    for (int[] p : pts) counts.merge(p, 1, Integer::sum);
    int max = Collections.max(counts.values());
    return counts.size() + " distinct, most repeated " + max + "x";
}
```

Test: countPoints({1,2}, {3,4}, {1,2}) → **Expected:** `2 distinct, most repeated 2x` **Got:** `3 distinct, most repeated 1x`

Solution 12.2 Counting Points

Why it fails. Java arrays compare by identity (where they live in memory), not by their contents. The two {1, 2} arrays are different objects, so the map treats them as different keys and every point is counted once.

✓ **Fixed code (changed lines are marked ✓)**

```
static String countPoints(int[][] pts) {
    Map<String, Integer> counts = new HashMap<>();
    for (int[] p : pts) counts.merge(p[0] + "," + p[1], 1, Integer::sum);   // ✓ value-based key
    int max = Collections.max(counts.values());
    return counts.size() + " distinct, most repeated " + max + "x";
}
```

Remember: Use a key whose equals() compares values: a String like "1,2", a List, or a record. Never use int[] as a map key.

Problem 12.3 Hand-Built Hash Table

Ignoring collisions · Hard

Task. A tiny hash table has 5 slots and stores keys at index key % 5. Keys 3 and 8 both map to slot 3. After put(3, "A") and put(8, "B"), get(3) must return A and get(8) must return B.

✗ **Buggy code. Find the bug before you read the solution.**

```
static String[] table = new String[5];
static void put(int k, String v) { table[k % 5] = v; }
static String get(int k) { return table[k % 5]; }
```

Test: put(3,"A"); put(8,"B"); get(3); get(8) → **Expected:** `A B` **Got:** `B B`

Solution 12.3 Hand-Built Hash Table

Why it fails. Keys 3 and 8 land in the same slot (3 % 5 == 8 % 5 == 3). The second put silently overwrites the first, so key 3 now returns B. Collisions are normal in hashing, and ignoring them causes silent data loss.

✓ **Fixed code (changed lines are marked ✓)**

```
static List<List<String[]>> table = new ArrayList<>();   // ✓ each slot is a bucket (list)
static { for (int i = 0; i < 5; i++) table.add(new ArrayList<>()); }
static void put(int k, String v) {
    for (String[] e : table.get(k % 5))
        if (e[0].equals("" + k)) { e[1] = v; return; }
    table.get(k % 5).add(new String[]{"" + k, v});
}
static String get(int k) {
    for (String[] e : table.get(k % 5))
        if (e[0].equals("" + k)) return e[1];
    return null;
}
```

Remember: Each slot must hold a bucket (a list of key-value pairs). On get/put, search the bucket and compare the actual keys.

REVISION

Debugging checklist: ask before you submit

When your code is wrong, times out or crashes, do not re-derive the fix under pressure. Find the matching row and run through the questions.

Part	Topic	Ask yourself
0	**Six Meta-Patterns Behind Almost Every Bug**	Did I say what should win, trace n = 0, 1, 2, and save a value before overwriting it?
1	**Arrays & Strings**	What happens for [] or "" and at the last index? Can the sum overflow int? Am I changing the input?
2	**Two Pointers & Sliding Window**	Does every branch move a pointer? Do I undo state when left moves and reset it per window?
3	**Binary Search**	Do low and high always move past mid? Is it lo <= hi? Is the data sorted the way my comparison assumes?
4	**Sorting**	What does my comparator return on ties? Can x - y overflow?
5	**Linked Lists**	Did I save next first? Did I handle null, one node, deleting the head and the tail pointer?
6	**Stacks & Queues**	Which end do I push and pop? Is the structure empty before I pop? Which operand pops first?
7	**Trees & Binary Search Trees**	Is the first line a null check? Do I need pre-, in- or post-order? Is a leaf height 0 or 1?
8	**Graphs (BFS/DFS)**	Did I mark visited when enqueuing (including the start)? Right edge direction? Fresh visited per call? Too deep for recursion?
9	**Dynamic Programming**	What does dp[0] mean? Is the loop direction right? Min or max? Is every result stored?
10	**Recursion & Backtracking**	Did I un-choose, copy before storing, prune early and pass i or i + 1?
11	**Greedy Algorithms**	Can I prove the greedy choice? Is the sort key right? Does touching count as overlap?
12	**Hashing & Hash Maps**	Did I seed the base case, use containsKey, avoid changing the map while iterating, and use a value-based key?

Search by shape, not by topic

When a fix does not jump out, run through the **six meta-patterns** of Part 0 before searching the topic tables: off-by-one, wrong comparison direction, missing or wrong base case, state not reset or updated, reference instead of snapshot, and order of operations. **Off-by-one** and **wrong comparison direction** alone account for a large share of every category.

THE GOLDEN RULE

Say out loud what the correct value **should be** at the smallest possible input. Find the exact line where the code **diverges** from that sentence. That line is almost always the bug.